NUZUL

Project Reference Document

Real Estate Management Platform for University Student Housing

Version 0.1.0 | April 2026

1. Project Overview

Nuzul is a full-stack web platform that connects university students looking for housing with landlords who manage properties near universities.

Core Features (Planned)

- Browse and filter available property listings
- Student registration and profile management
- Landlord property listing management (create, update, delete)
- Booking requests from students to landlords
- Authentication via Email/Password and Google OAuth

Current Status: Project base is fully scaffolded and running. Backend API is live with all endpoints. Frontend renders with routing and auth wired up. Database tables need to be created in Supabase SQL Editor.

---

2. Tech Stack

Layer	Technology	Version
Backend Framework	FastAPI	0.135+
Backend Language	Python	3.11
Backend Server	Uvicorn	0.44+
Auth Provider	Supabase Auth	—

Database	PostgreSQL (Supabase-hosted)	—
Supabase Client	supabase-py	2.28+
Frontend Framework	React	18
Frontend Language	TypeScript	5
Frontend Build Tool	Vite	8
Styling	Tailwind CSS	v4
Routing	React Router	v6
State Management	Zustand	4
HTTP Client	Axios	1
Containerization	Docker + Docker Compose	—

---

3. Repository Structure

The project uses a monorepo — one Git repository containing both backend and frontend.

```
Nuzul/  (monorepo root)
```

Backend — backend/

Path	Description
app/main.py	App entry point, middleware, router registration, lifespan
app/api/routers/auth.py	Auth endpoints: /api/v1/auth/*
app/api/routers/properties.py	Property endpoints: /api/v1/properties/*
app/api/routers/bookings.py	Booking endpoints: /api/v1/bookings/*
app/core/config.py	Environment settings loaded via pydantic-settings
app/core/database.py	Supabase client singletons (service-role + anon)
app/core/security.py	JWT verification via Supabase Auth
app/models/	SQLModel models (schema reference — not used at runtime)
app/schemas/	Pydantic schemas (request/response documentation)
alembic/	DB migrations (not active — tables created via SQL Editor)
requirements.txt	Python dependencies
.env.example	Environment variable template
Dockerfile	Container definition

Frontend — frontend/

Path	Description
src/main.tsx	React entry point
src/App.tsx	Router setup and auth bootstrap
src/index.css	Tailwind v4 import
src/lib/supabase.ts	Supabase JS client (anon key)
src/services/api.ts	Axios instance with automatic JWT header injection
src/services/auth.ts	Auth service: sign in, sign up, Google OAuth
src/services/properties.ts	Properties API calls
src/hooks/useAuth.ts	Auth state hook and bootstrap logic
src/store/authStore.ts	Zustand auth store (session, user, isLoading)
src/types/index.ts	Shared TypeScript interfaces
src/components/layout/Navbar.tsx	Top navigation with auth state
src/components/layout/ProtectedRoute.tsx	Route guard for authenticated pages
src/components/ui/	Empty — ready for shared UI components
src/pages/LoginPage.tsx	Email + Google sign in
src/pages/RegisterPage.tsx	Email + Google sign up
src/pages/AuthCallbackPage.tsx	Google OAuth redirect handler
src/pages/ListingsPage.tsx	Browse available properties
src/pages/DashboardPage.tsx	User dashboard (placeholder)
vite.config.ts	Vite config with @ alias and API proxy

---

4. Architecture

Layer	Flow
1. User Action	User logs in via Supabase (email or Google OAuth)
2. Token Issued	Supabase Auth issues a JWT access token to the browser
3. API Request	Frontend attaches JWT in Authorization: Bearer header via Axios interceptor
4. JWT Verification	FastAPI backend calls Supabase Auth get_user(token) to verify

5. DB Operation	Backend uses service-role Supabase client to read/write database
6. Response	Data returned as JSON to the frontend

Key Design Decision: The backend does NOT connect to PostgreSQL directly. It uses the Supabase Python client (supabase-py) which communicates via Supabase REST API. This eliminates DATABASE_URL requirements and works on any network.

---

5. Backend — Deep Dive

5.1 Entry Point — app/main.py

- Creates FastAPI app with lifespan context
- On startup: pings the users table to verify Supabase connection
- Registers CORS middleware (allows localhost:5173 and localhost:3000)
- Mounts all routers under /api/v1
- Exposes /health endpoint (no auth required)
- /docs and /redoc available in debug mode only

5.2 Config — app/core/config.py

Uses pydantic-settings to load environment variables from .env. Cached with @lru_cache so Settings is only instantiated once.

Required Environment Variables

Variable	Description
SUPABASE_URL	Your Supabase project URL
SUPABASE_ANON_KEY	Public anon key for auth token verification
SUPABASE_SERVICE_ROLE_KEY	Secret key for database operations (bypasses RLS)

Optional Environment Variables (have defaults)

Variable	Default	Description
APP_NAME	Nuzul	Application name
APP_ENV	development	Environment mode
DEBUG	true	Enable debug mode and API docs

ALLOWED_ORIGINS	localhost:5173, localhost:3000	CORS allowed origins

5.3 Database — app/core/database.py

Client	Function	Usage
get_db()	Service-role singleton	All DB read/write operations. Bypasses RLS. NEVER expose to frontend.
get_auth_client()	Fresh anon-key instance	JWT token verification only. New instance each call to prevent session bleeding.

5.4 Security — app/core/security.py

Dependency	Description
verify_supabase_token	Extracts Bearer token, calls Supabase auth.get_user(), returns { id, email, user_metadata }. Raises 401 on failure.
get_current_user_id	Shortcut wrapping verify_supabase_token. Returns just the Supabase user UUID string.

---

6. API Reference

Base URL (dev): http://localhost:8000 **API Prefix:** /api/v1

Auth Header: Authorization: Bearer <supabase_access_token>

Health

Method	Path	Auth	Description
GET	/health	None	Returns app status, env, and Supabase connection status

Auth Endpoints — /api/v1/auth

Method	Path	Auth	Description
POST	/register	JWT	Creates user row in users table after Supabase signup. Idempotent — safe to call multiple times.

GET	/me	JWT	Returns current user profile from users table.
PATCH	/me	JWT	Updates full_name or avatar_url fields only.

Property Endpoints — /api/v1/properties

Method	Path	Auth	Description
GET	/	None	List properties. Supports filters: city, min_price, max_price, rooms, available_only.
GET	/{id}	None	Get single property by UUID.
POST	/	JWT	Create new property. Sets owner_id from authenticated user.
PATCH	/{id}	JWT	Update property. Only the owner can update.
DELETE	/{id}	JWT	Delete property. Only the owner can delete. Returns 204.

Query Parameters for GET /api/v1/properties

Parameter	Type	Default	Description
city	string	—	Filter by city name
min_price	float	—	Minimum price per month
max_price	float	—	Maximum price per month
rooms	integer	—	Number of rooms
available_only	boolean	true	Show only available properties

Booking Endpoints — /api/v1/bookings

Method	Path	Auth	Description
GET	/my	JWT	List all bookings made by the current student.
POST	/	JWT	Create booking request. Sets student_id from auth user, status defaults to pending.
PATCH	/{id}	JWT	Update booking. Only the student who created it can update. Allowed: status, move_out_date.

---

7. Database Schema

Tables must be created manually in the Supabase SQL Editor. Run the SQL below in your Supabase project.

users

Column	Type	Default	Notes
id	UUID	gen_random_uuid()	Primary key
supabase_id	TEXT	—	UNIQUE. Maps to Supabase auth.users.id
email	TEXT	—	UNIQUE
full_name	TEXT	NULL	Optional
avatar_url	TEXT	NULL	Optional
role	TEXT	student	Values: student, landlord, admin
is_active	BOOLEAN	true	—
created_at	TIMESTAMPTZ	now()	—
updated_at	TIMESTAMPTZ	now()	—

properties

Column	Type	Default	Notes
id	UUID	gen_random_uuid()	Primary key
owner_id	UUID	—	FK → users.id ON DELETE CASCADE
title	TEXT	—	Required
description	TEXT	NULL	Optional
address	TEXT	—	Required
city	TEXT	—	Required
price_per_month	NUMERIC	—	Required
rooms	INTEGER	1	—
is_available	BOOLEAN	true	—
near_university	TEXT	NULL	University name or distance tag
image_url	TEXT	NULL	Optional
created_at	TIMESTAMPTZ	now()	—
updated_at	TIMESTAMPTZ	now()	—

bookings

Column	Type	Default	Notes

id	UUID	gen_random_uuid()	Primary key
property_id	UUID	—	FK → properties.id ON DELETE CASCADE
student_id	UUID	—	FK → users.id ON DELETE CASCADE
status	TEXT	pending	Values: pending, confirmed, cancelled
move_in_date	TIMESTAMPTZ	—	Required
move_out_date	TIMESTAMPTZ	NULL	Optional
message	TEXT	NULL	Optional note from student
created_at	TIMESTAMPTZ	now()	—
updated_at	TIMESTAMPTZ	now()	—

---

8. Auth Flow

8.1 Email Sign Up

Step	Action
1	User fills RegisterPage form and submits
2	authService.signUpWithEmail() called → Supabase sends confirmation email
3	User clicks confirmation link in email
4	Supabase fires onAuthStateChange with SIGNED_IN event
5	useAuthBootstrap calls POST /api/v1/auth/register (creates row in users table)
6	useAuthBootstrap calls GET /api/v1/auth/me (fetches user profile)
7	User profile saved to Zustand store — app shows authenticated state

8.2 Google OAuth

Step	Action
1	User clicks 'Continue with Google'
2	authService.signInWithGoogle() redirects to Google consent screen
3	Google redirects to /auth/callback with session tokens in URL
4	AuthCallbackPage reads session and redirects to /dashboard
5	Same as steps 4-7 of Email Sign Up above

8.3 Every API Request

Step	Action
1	Axios request interceptor runs before every API call
2	Fetches current Supabase session from local storage
3	Attaches Authorization: Bearer <access_token> header
4	FastAPI security.py calls Supabase auth.get_user(token)
5a	If valid → route handler executes and returns data
5b	If invalid/expired → 401 Unauthorized returned, user signed out

---

9. Frontend — Deep Dive

9.1 Routing

Path	Auth	Component	Description
/	Public	Redirect	Redirects to /listings
/listings	Public	ListingsPage	Browse available properties
/login	Public	LoginPage	Email + Google sign in
/register	Public	RegisterPage	Create account
/auth/callback	Public	AuthCallbackPage	OAuth redirect handler
/dashboard	Protected	DashboardPage	User dashboard
*	Public	Redirect	Redirects to /listings

9.2 Auth System

File	Purpose
src/lib/supabase.ts	Creates Supabase JS client. Reads VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY.
src/store/authStore.ts	Zustand store. Holds: session, user (backend profile), isLoading.
src/hooks/useAuth.ts	useAuthBootstrap() — call ONCE at App root. useAuth() — use anywhere to read auth state.
src/components/layout/ProtectedRoute.tsx	Shows loading spinner while auth bootstraps. Redirects to /login if no session.

9.3 TypeScript Types

Type	Key Fields
User	id, supabase_id, email, full_name, avatar_url, role, is_active, created_at
Property	id, owner_id, title, description, address, city, price_per_month, rooms, is_available, near_university, image_url
Booking	id, property_id, student_id, status, move_in_date, move_out_date, message, created_at
PropertyCreate	Omit<Property, id | owner_id | created_at | updated_at>
PropertyUpdate	Partial<PropertyCreate>
BookingCreate	Pick<Booking, property_id | move_in_date | move_out_date | message>
BookingUpdate	Partial<Pick<Booking, status | move_out_date>>

10. Environment Variables

Backend — backend/.env

Variable	Value / Where to Find
APP_NAME	Nuzul
APP_ENV	development
DEBUG	true
SUPABASE_URL	Supabase → Project Settings → API → Project URL
SUPABASE_ANON_KEY	Supabase → Project Settings → API Keys → anon/public
SUPABASE_SERVICE_ROLE_KEY	Supabase → Project Settings → API Keys → service_role
ALLOWED_ORIGINS	http://localhost:5173,http://localhost:3000

Frontend — frontend/.env

Variable	Value / Where to Find
VITE_SUPABASE_URL	Same as backend SUPABASE_URL
VITE_SUPABASE_ANON_KEY	Same as backend SUPABASE_ANON_KEY
VITE_API_BASE_URL	http://localhost:8000 (dev) or deployed backend URL (prod)

Security: Never commit .env files to Git. Both .env files are excluded in .gitignore. The SUPABASE_SERVICE_ROLE_KEY is backend-only — never use it in the frontend.

11. Running the Project

11.1 Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate       # Mac/Linux
pip install fastapi[standard] sqlmodel asyncpg alembic
python-jose[cryptography] supabase
copy .env.example .env            # fill in your Supabase values
uvicorn app.main:app --reload --port 8000
```

API Docs: http://localhost:8000/docs

11.2 Frontend

```
cd frontend
npm install
copy .env.example .env            # fill in your Supabase values
npm run dev
```

App: http://localhost:5173

11.3 Create Database Tables

Go to Supabase → SQL Editor and run the following SQL:

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  supabase_id TEXT UNIQUE NOT NULL,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  role TEXT DEFAULT 'student' CHECK (role IN ('student', 'landlord',
'admin')),
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

```
CREATE TABLE properties (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id UUID REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  address TEXT NOT NULL,
  city TEXT NOT NULL,
  price_per_month NUMERIC NOT NULL,
  rooms INTEGER DEFAULT 1,
  is_available BOOLEAN DEFAULT true,
  near_university TEXT,
  image_url TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE bookings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_id UUID REFERENCES properties(id) ON DELETE CASCADE,
  student_id UUID REFERENCES users(id) ON DELETE CASCADE,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed',
'cancelled')),
  move_in_date TIMESTAMPTZ NOT NULL,
  move_out_date TIMESTAMPTZ,
  message TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

---

12. What's Built vs What's TODO

Done

The following items are fully implemented and working.

Backend

- FastAPI app with CORS, lifespan, and health check endpoint
- Supabase auth integration (JWT verification via get_user)
- Auth router: /register, /me (GET), /me (PATCH)
- Properties router: full CRUD with owner authorization
- Bookings router: create, list, update with student authorization
- Service-role DB client and anon auth client (database.py)

- Environment config with pydantic-settings
- Docker and Docker Compose setup

Frontend

- Vite + React 18 + TypeScript project setup
- Tailwind CSS v4 with @ path alias
- React Router v6 with protected routes
- Supabase JS client (lib/supabase.ts)
- Axios with automatic JWT injection (services/api.ts)
- Zustand auth store (store/authStore.ts)
- Auth bootstrap hook (hooks/useAuth.ts)
- Login page: email + Google OAuth
- Register page: email + Google OAuth with confirmation screen
- Auth callback page for OAuth redirect
- Listings page with data fetching wired up
- Dashboard page (placeholder with stat cards)
- Navbar with auth state
- TypeScript types for all entities

TODO — Backend

- Add Pydantic request validation to routers (replace raw dict)
- Add pagination to GET /properties
- Add landlord role check on property creation
- Add landlord booking management (confirm/cancel bookings)
- Add GET /properties/my for landlord's own listings
- Add Supabase Storage integration for property images
- Add input validation and better error messages
- Write unit and integration tests

TODO — Frontend

- Style all pages properly with Tailwind
- Property detail page
- Property creation form (for landlords)
- Booking request form
- User profile/settings page
- Dashboard with real booking data
- Search and filter UI on listings page
- Image upload for property photos
- Error boundary and loading states
- Mobile responsive layout

- Bookings service (src/services/bookings.ts)